

Learned Indexes vs B+ Trees

Case study explained in simple words, and what our implementation shows

1. What your case study is about

The problem. A database index is like a book's index: it helps you find a record without scanning everything. For about 50 years, the standard has been the B+ Tree, a tree of nodes that you walk down level by level, following pointers. That worked well when data lived on disk. Now that data lives in RAM, following many pointers is slow (CPU cache misses), and the tree's internal nodes use a lot of memory.

The new idea. The Kraska et al. 2018 paper said that an index is really just a function: give it a key, and it returns the position in a sorted array. If you can learn the shape of your data (its CDF, which is just "what fraction of keys are smaller than this one?"), you can calculate the position instead of searching for it. Compare finding a word in a dictionary by flipping to roughly 70% of the way through and adjusting, versus starting at the middle and halving each time. The model makes the first guess, and a small error bound tells you how far to look around it.

Your report's conclusion. Learned indexes are not a universal replacement. They are a specialized tool:

- **Win:** much less memory (your report says up to about 95% less) and faster lookups (40–150% faster), especially on predictable, smooth data.
- **Lose:** inserts are 10–50x slower, because new data changes the shape the model learned. They also struggle on messy, clustered data.

2. How the research papers fit together

Think of the four papers as a story:

Paper	Role in the story
Kraska 2018 (RMI)	The idea. Models can replace trees, but only for read-only data.
Ding 2020 (ALEX)	The fix for inserts. Leave gaps in the array so new keys fit without retraining everything.
Ferragina 2020 (PGM)	The guarantee. Approximate the data with straight-line segments and a provable maximum error, so performance is predictable.
Marcus 2020 (SOSD)	The referee. A standard way to compare them fairly across datasets.

3. What we add or learn

Your report is a comparative evaluation, so the contribution isn't a new index. It is:

1. Putting the papers' claims side by side in one controlled experiment.
2. Testing when each claim holds (which data distribution, which workload).
3. Turning the findings into practical advice ("use a learned index if your data is X and mostly read, otherwise a B+ Tree").

4. What our implementation shows

Our Python benchmark is a small, reproducible re-test of your report's main claims:

Your report's claim	What we measure
Learned indexes use far less memory	Memory footprint of B-tree vs RMI vs PGM
Learned indexes have faster lookups	Lookup latency across datasets
Inserts are much slower for learned indexes	Insert throughput, plus retraining cost
Delta buffers cause inconsistent performance	Retrain count and time
Smooth data suits models, chaotic data doesn't	Uniform vs lognormal vs clustered datasets, plus the CDF plots
Index choice depends on the workload	Mixed read/write workloads (100/0, 90/10, 50/50)
PGM gives predictable error bounds	The ϵ sweep (accuracy vs size trade-off)

5. One honest caveat to state in your presentation

Your report's numbers come from C++ on a Xeon server. Ours come from Python on a laptop, so **absolute speeds won't match**. The memory savings and the pattern (learned wins on smooth data and reads, B+ Tree wins on writes and messy data) should still show up.

In Python, the lookup speed advantage may even be weaker or reversed, because interpreter overhead hides the cache effects. If that happens, it is a legitimate finding: say so, and explain that the cache-miss advantage needs low-level code to appear.